Work Integrated Learning Programmes
M.Tech Software Engineering



Scalable Services

Assignment 1

Microservices Design and Development Report

Movie Ticket Booking System

By: Group 9

Name	BITS ID
Gujar Neel Lukesh	2025tm93111
Yamuna P Hegade	2025tm93019
Bhandari Akshay Sunil	2025tm93112
Naveen Venkat P	2025tm93528

1. Problem and Scope

1.1 Objective

The objective of this assignment is to design and implement a scalable Movie Ticket Booking System using a microservices architecture. The system demonstrates key distributed systems concepts such as service decomposition, database-per-service design, concurrency control, idempotent request handling, and Saga-based transaction management.

The implemented application consists of five independent microservices:

- User Service
- Show Service
- Booking Service
- Payment Service
- Notification Service

The system supports end-to-end movie ticket booking workflows while ensuring consistency, fault tolerance, and scalability.

1.2 Problem Statement

The system allows users to book movie tickets while handling real-world distributed systems challenges.

Contended Resource

- Seats

Cross-Service Transaction

- Book Seat

  → Reserve Seat

  → Payment

  → Confirmation

  → Notification

Unsafe Operation

- Payment Processing

Mock Failing Dependency

- Payment Gateway

Challenges Implemented

- Challenge 1: Concurrency

- Challenge 2: Idempotency
- Challenge 3C: Saga Compensation

1.3 Assumptions

1. All services run locally and communicate using REST APIs.
2. SQLite databases are used for persistence.
3. Payment Gateway is simulated.
4. Notification delivery is mocked and stored for verification.
5. Services communicate synchronously over HTTP.

1.4 Business Rules

- BR1: A seat can only be reserved once.
- BR2: A seat must be AVAILABLE before reservation.
- BR3: Only ACTIVE users are allowed to create bookings.
- BR4: Booking can be CONFIRMED only after successful payment.
- BR5: Duplicate payment requests with the same idempotency key are processed exactly once.
- BR6: Payment failure triggers booking cancellation and seat release.

1.5 Out of Scope

- Real payment gateway integration
- Email/SMS provider integration
- Authentication and authorization
- Distributed deployment
- Kubernetes orchestration
- Production-grade monitoring

---

2. Service Decomposition

Responsibility Table

Service	Responsibility
User Service	**User registration and lifecycle management**
Show Service	**Show and seat management**
Booking Service	**Booking lifecycle and Saga orchestration**
Payment Service	**Payment processing and idempotency**
Notification Service	**User notifications**

Each service owns a single business capability and database. This separation reduces coupling, improves maintainability, and allows independent deployment and scaling.

Alternatives Considered

Alternative 1 – Monolith

Rejected because:

A monolithic system would not demonstrate distributed-systems concepts and would limit scalability.

Alternative 2 – Booking + Payment Combined

Rejected because:

Payment processing requires separate idempotency logic and business rules.

---

3. System Architecture

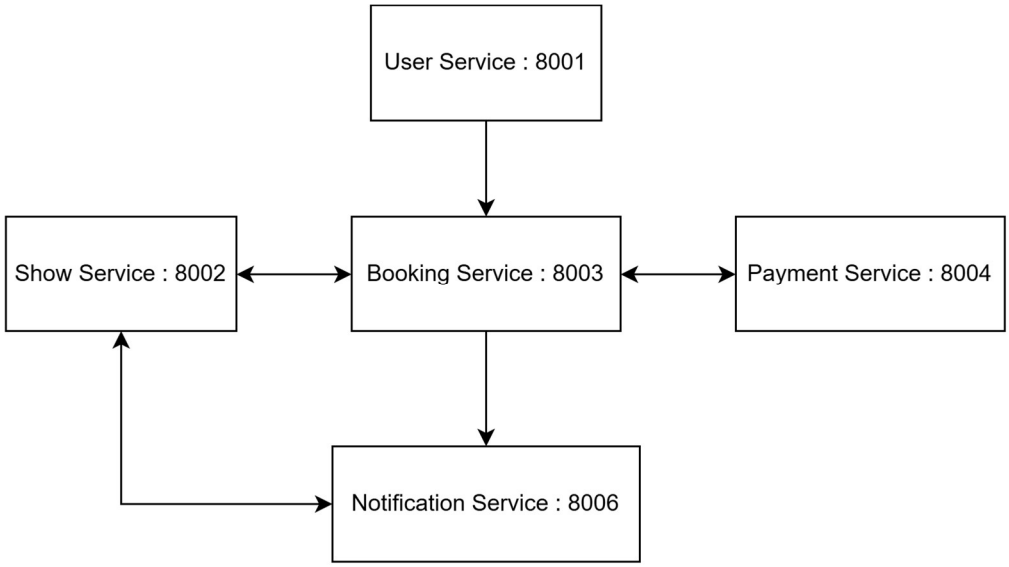


Figure 1. Microservices Architecture of the Movie Ticket Booking System

Data Ownership

Service	Database Ownership
User Service	**Users**
Show Service	**Shows, Seats**
Booking Service	**Bookings**
Payment Service	**Payments**
Notification Service	**Notifications**

Cross-Service References

Services do not share databases. References such as user_id and show_id are stored locally while validation occurs through API communication.

Description

The system follows the microservices pattern where each business capability is implemented as an independent service.

Service	Responsibility
User Service	Manages users and account status
Show Service	Manages shows and seat inventory
Booking Service	Handles bookings and Saga orchestration
Payment Service	Processes payments and idempotency
Notification Service	Sends booking notifications

Service Ports

Service	Port
User Service	8001
Show Service	8002
Booking Service	8003
Payment Service	8004
Notification Service	8006

Sequence Diagrams

Successful Booking Flow

User
|
Create Booking
v
Booking Service
|
Reserve Seat
v
Show Service
|
Success
v
Booking Service
|
Process Payment

v
Payment Service
|
Success
v
Booking Service
|
Confirm Booking
|
Send Notification
v
Notification Service
|
v
Booking Confirmed

Steps:

1. User creates booking.
2. Booking Service reserves seat.
3. Payment Service processes payment.
4. Booking is confirmed.
5. Notification is generated.

Failed Booking Flow

User
|
Create Booking
v
Booking Service
|
Reserve Seat
v
Show Service
|
Success
v
Booking Service
|
Process Payment
v
Payment Service

|
Failure
v
Booking Service
|
Release Seat
v
Show Service
|
Seat Released
v
Booking Service
|
Cancel Booking
|
Send Notification
v
Notification Service
|
v
Booking Cancelled

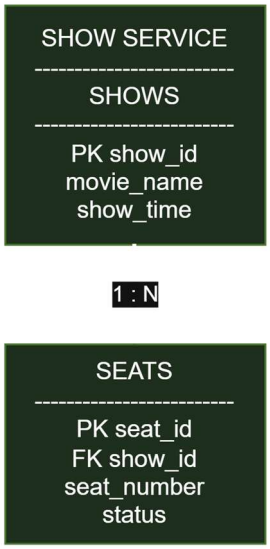
---

4. Data Design

USER SERVICE



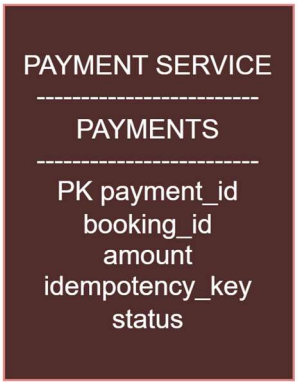
SHOW SERVICE



BOOKING SERVICE



PAYMENT SERVICE



NOTIFICATION SERVICE



Database-per-Service Pattern

The solution follows the **Database-per-Service** architectural pattern. Each microservice owns and manages its own database and no service directly accesses another service's data store. Communication between services is performed exclusively through REST APIs. This approach reduces coupling, preserves service autonomy, and improves maintainability.

Data Ownership Table

Service	Data Owned
User Service	Users
Show Service	Shows, Seats
Booking Service	Bookings
Payment Service	Payments
Notification Service	Notifications

Cross-Service References

The system maintains logical relationships between services using identifiers rather than direct database joins.

Reference	Purpose
user_id	Links bookings to users
show_id	Links bookings to shows
booking_id	Links payments and notifications to bookings

All validation and data retrieval involving these references are performed through service APIs rather than database access.

Status Models

Show Service Seat States

- AVAILABLE
- RESERVED
- BOOKED

Booking Service States

- PENDING
- RESERVED
- CONFIRMED
- PAYMENT_FAILED
- CANCELLED

These states are used to enforce business rules, support concurrency control, and implement the Saga workflow.

Seed Data Used During Integration Testing

The integration environment was initialized using the following sample dataset:

Show

- Show ID: 1
- Movie: Demo Movie
- Time: 18:00

Seats

- A1
- A2
- A3
- A4
- A5

All seats were initially configured with status:

AVAILABLE

This dataset was used for concurrency testing, Saga validation, and end-to-end integration verification.

---

5. API Design

Endpoint Summary

Service	Method	Endpoint
User	**POST**	**/api/v1/users**
User	**GET**	**/api/v1/users/{id}**
Show	**GET**	**/shows**
Show	**POST**	**/seats/reserve**
Show	**POST**	**/seats/release**

Booking	**POST**	**/bookings**
Booking	**GET**	**/bookings/{id}**
Booking	**POST**	**/bookings/{id}/process**
Payment	**POST**	**/payments**
Notification	**POST**	**/notifications**

Error Model

HTTP Code	Meaning
400	**Business Rule Violation**
404	**Resource Not Found**
409	**Concurrency Conflict**
422	**Validation Error**
500	**Internal Error**

Example:

{

"detail": "Seat is not available for reservation"

}

5.1 User Service

Responsibilities:

- Register users
- Retrieve user information
- Activate/deactivate users
- Enforce account lifecycle constraints

APIs

- POST /api/v1/users
- GET /api/v1/users/{id}
- POST /api/v1/users/{id}/activate
- POST /api/v1/users/{id}/deactivate

Screenshots



Figure 2. User Creation API

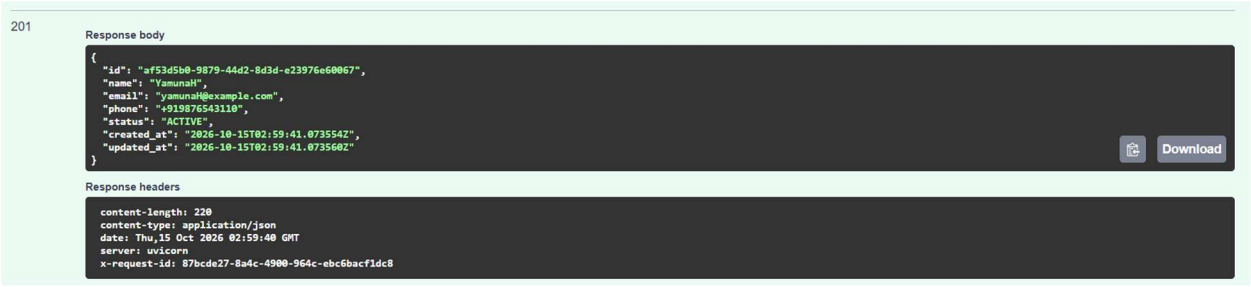


Figure 3. Successful User Retrieval

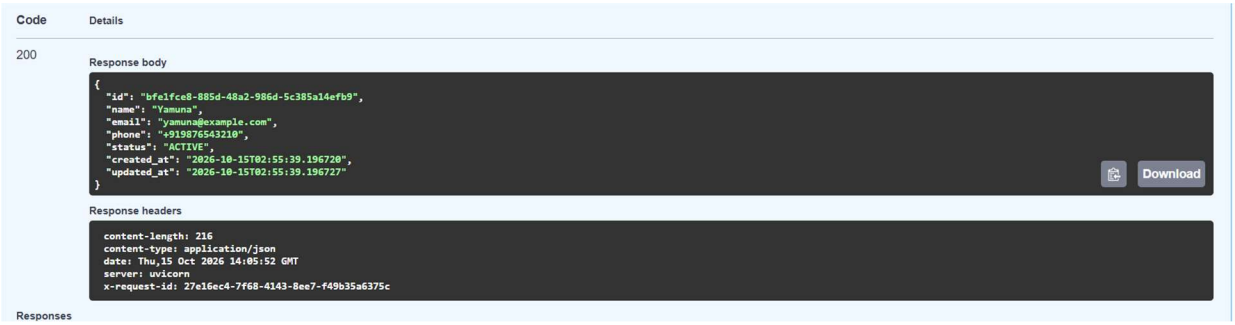


Figure 4. User Service Validation

5.2 Show Service

Responsibilities:

- Manage movie shows
- Manage seat inventory
- Reserve seats
- Release seats

APIs

- GET /shows
- POST /seats/reserve
- POST /seats/release

5.3 Booking Service

Responsibilities:

- Create bookings
- Maintain booking states
- Coordinate Saga workflow

APIs

- POST /bookings
- GET /bookings/{booking_id}
- PUT /bookings/{booking_id}/status
- POST /bookings/{booking_id}/process
- POST /reserve-seat

Booking State Machine

Success Flow:

PENDING

↓

RESERVED

↓

CONFIRMED

Failure Flow:

PENDING

↓

RESERVED

↓

PAYMENT_FAILED

↓

CANCELLED

5.4 Payment Service

Responsibilities:

- Process payments
- Enforce idempotency
- Support Saga compensation

API

- POST /payments

5.5 Notification Service

Responsibilities:

- Create notifications
- Retrieve notification history

OpenAPI Links

Swagger/OpenAPI Endpoints

User Service:

http://localhost:8001/docs

Show Service:

http://localhost:8002/docs

Booking Service:

http://localhost:8003/docs

Payment Service:

http://localhost:8004/docs

Notification Service:

http://localhost:8006/docs

All services expose automatically generated OpenAPI specifications through FastAPI Swagger interfaces, enabling interactive testing and API discovery.

Sample Request / Response

Sample Booking Request


```
{
 "user_id": "b4f6bbc7-ef74-41b8-86c1-c1fc787536e1",
 "show_id": 1,
 "seat_number": "A2"
}
```


Sample Booking Response


```
{
 "booking_id": 7,
 "status": "PENDING"
}
```


Sample Payment Request


```
{
 "idempotency_key": "PAY123"
}
```


Sample Payment Response


```
{
 "message": "Payment processed"
}
```


---

6. Engineering Challenges

Challenge 1 – Concurrency Control

Objective

Prevent multiple users from reserving the same seat simultaneously.

Problem

Multiple users may attempt to reserve the same seat at the same time, potentially causing double booking and inconsistent seat allocation.

Implementation

The Show Service performs atomic seat reservation by updating seats only when their current state is AVAILABLE.

Design Choice

Seat reservation is implemented using a single atomic database update operation that succeeds only when the seat status is AVAILABLE.

Alternative Considered

A distributed lock mechanism was considered. However, it was rejected because it would introduce additional complexity for an assignment-scale implementation.

Test Scenario

Twenty concurrent reservation attempts were executed against the same seat.

Screenshot

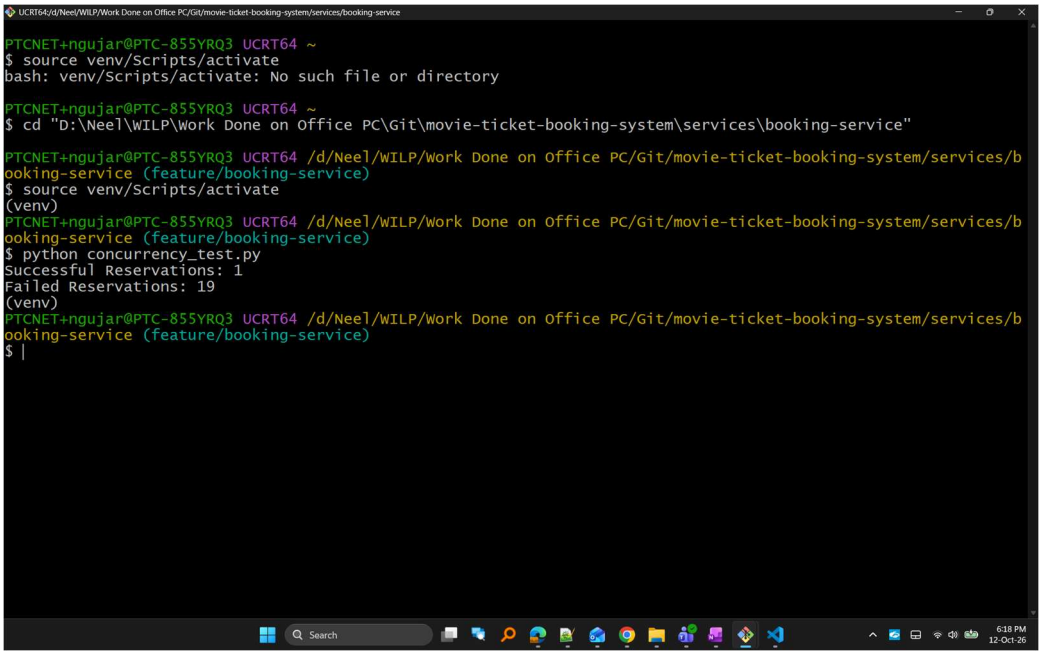


Figure 5. Concurrency Test Results

Results

Successful Reservations: 1

Failed Reservations: 19

Analysis

Only one request successfully reserved the seat while all remaining requests were rejected, demonstrating correct handling of a contended resource.

Limitation

The implementation uses SQLite and local execution for demonstration purposes. In a production environment, stronger concurrency mechanisms and distributed deployment strategies may be required.

Challenge 2 – Idempotency

Objective

Prevent duplicate payment processing.

Problem

Network retries or client resubmissions can result in the same payment request being sent multiple times.

Implementation

The Payment Service stores and validates idempotency keys.

Repeated requests using the same key return a duplicate response instead of creating a new payment record.

Design Choice

Each payment request contains a unique idempotency key which is stored and checked before processing.

Alternative Considered

Transaction fingerprinting based on payment payloads was considered but rejected because explicit idempotency keys provide a simpler and more reliable solution.

Screenshots

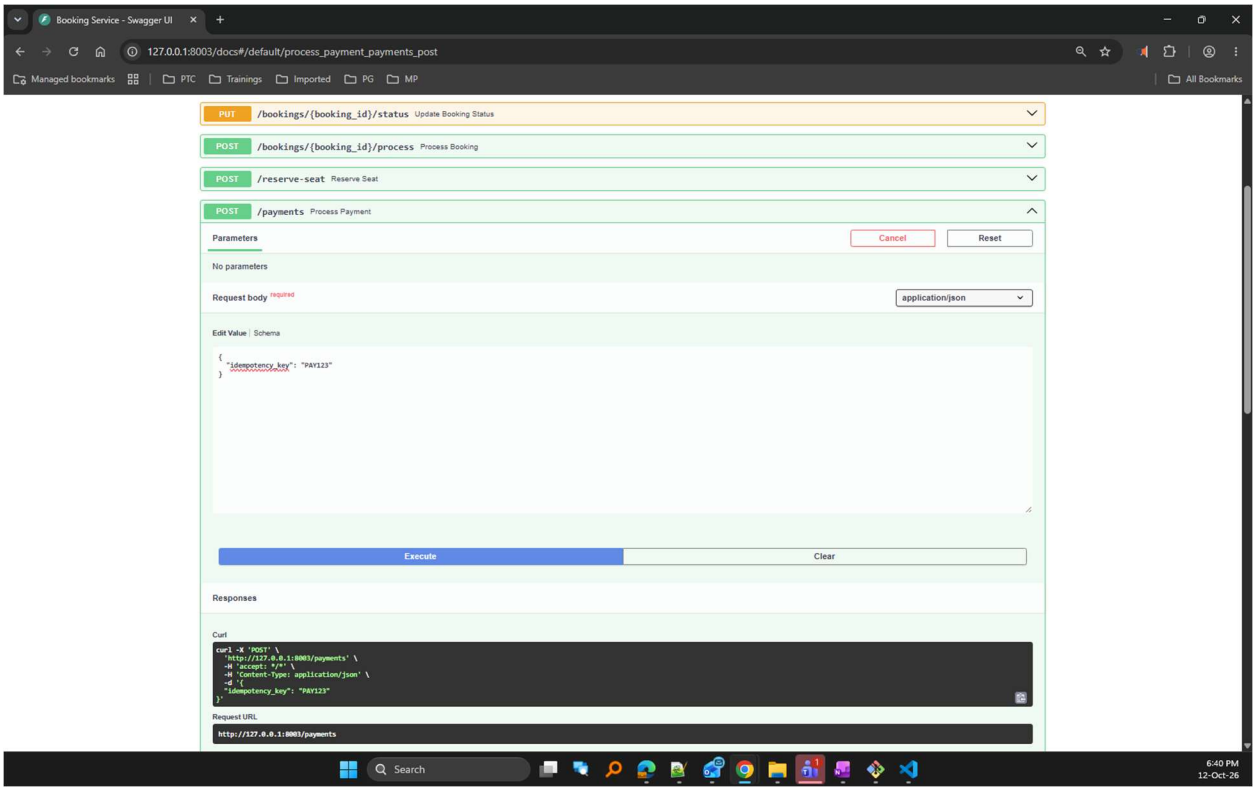


Figure 6. Initial Payment Processing

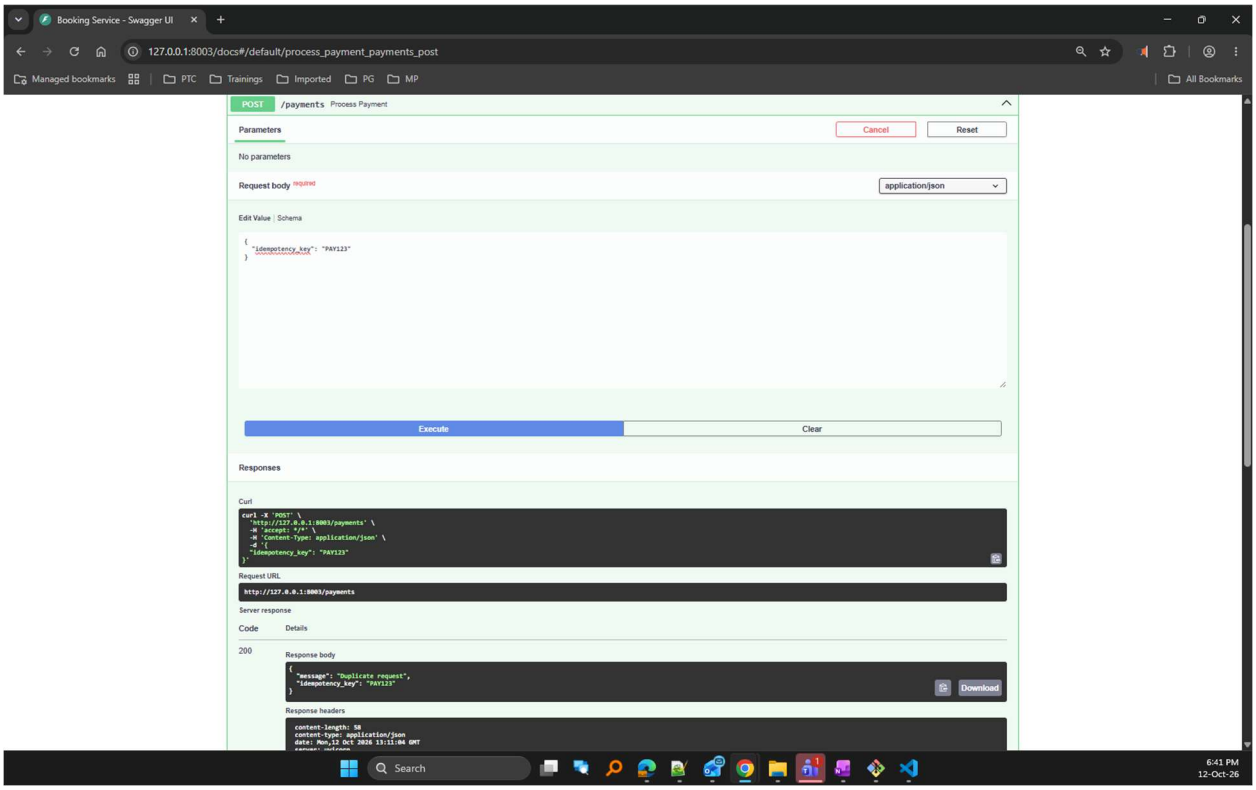
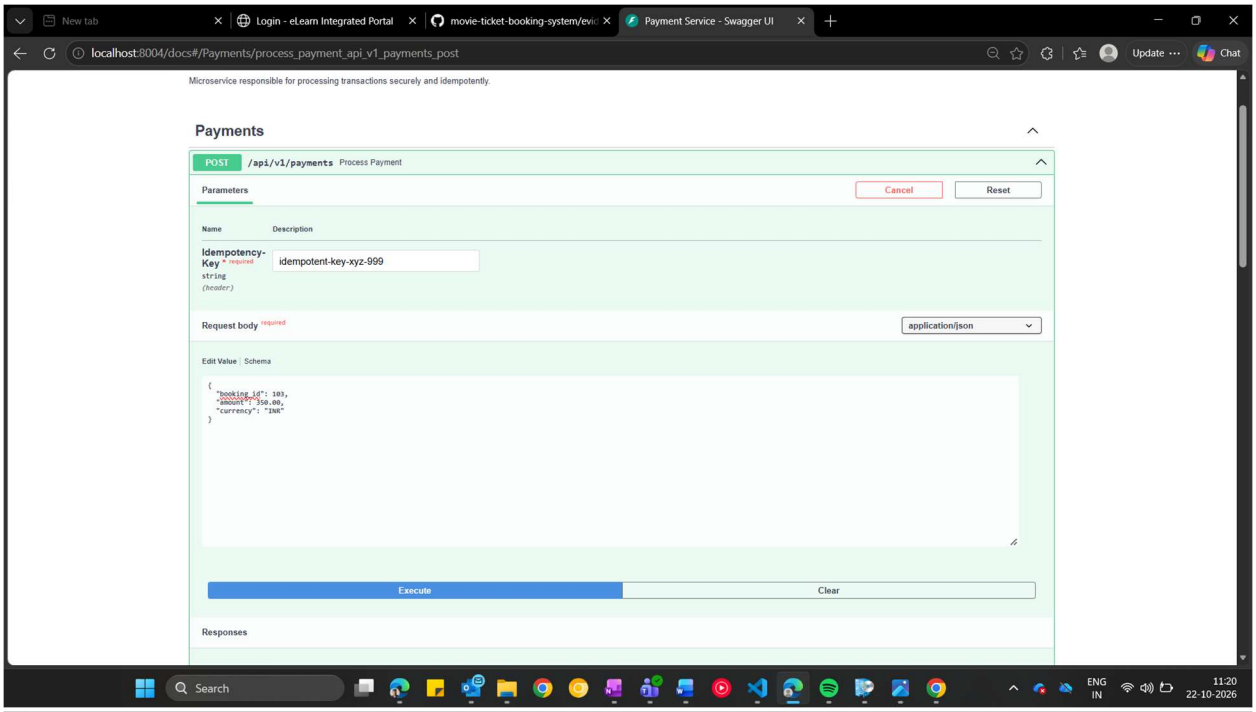
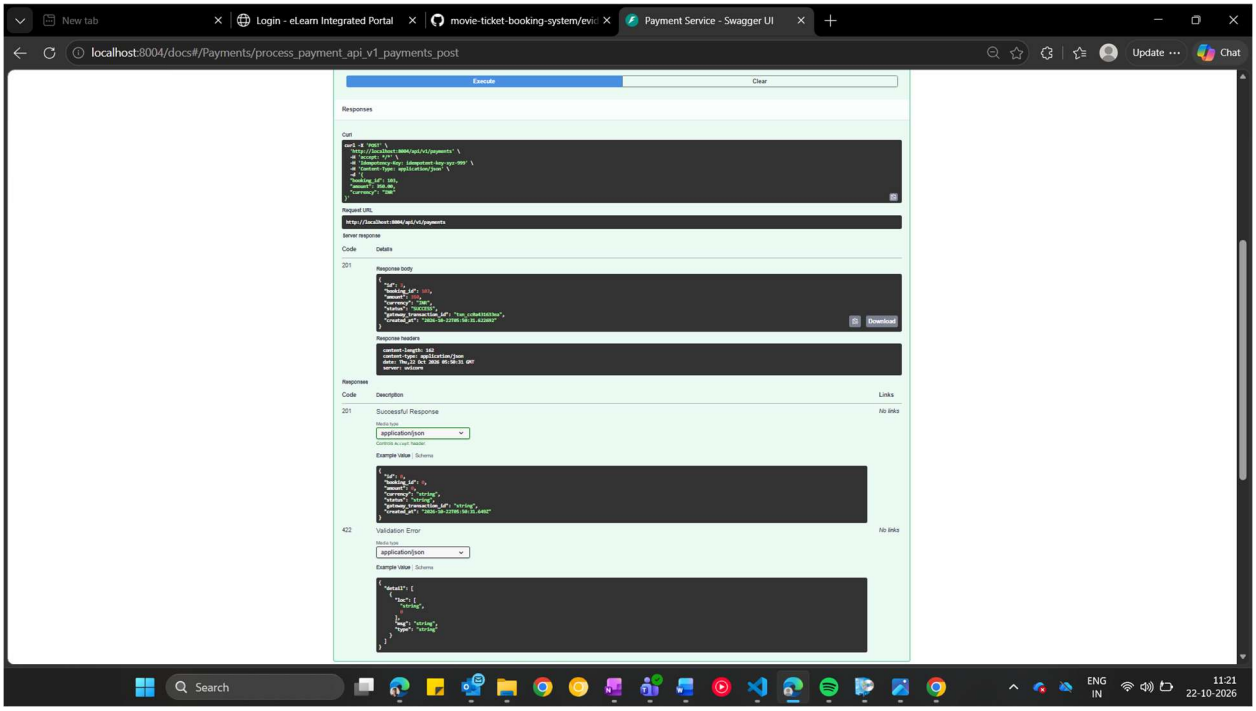
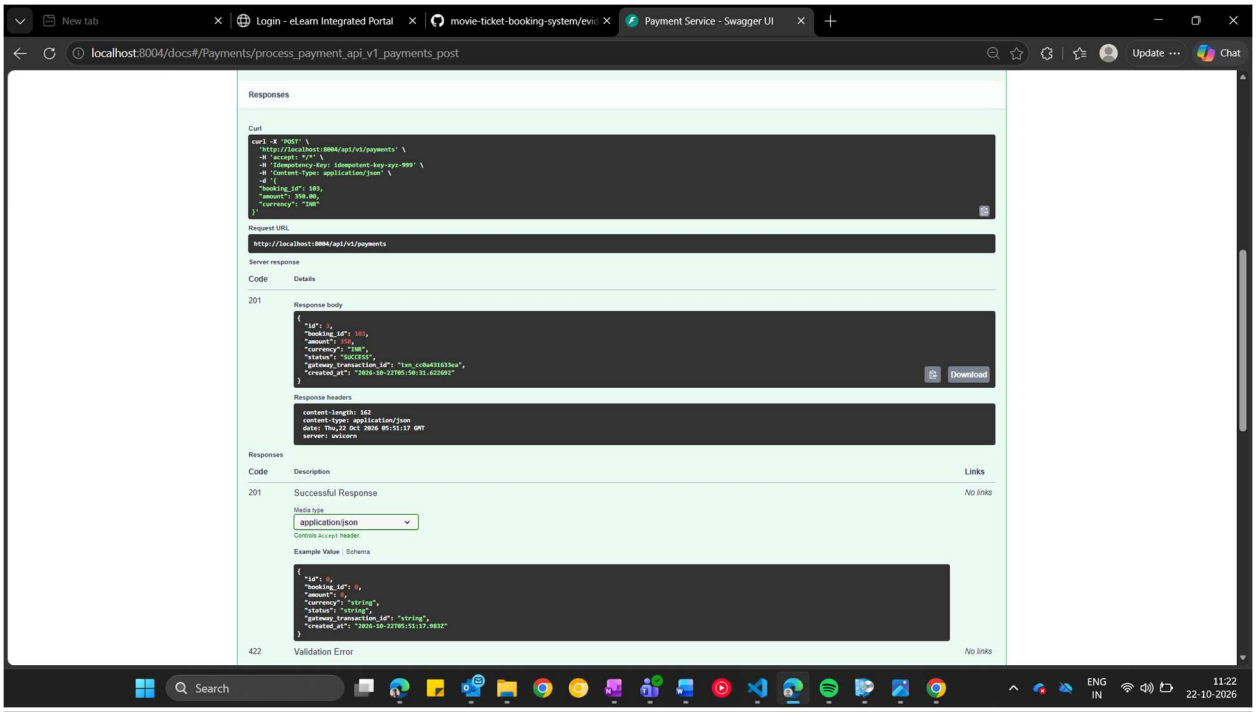
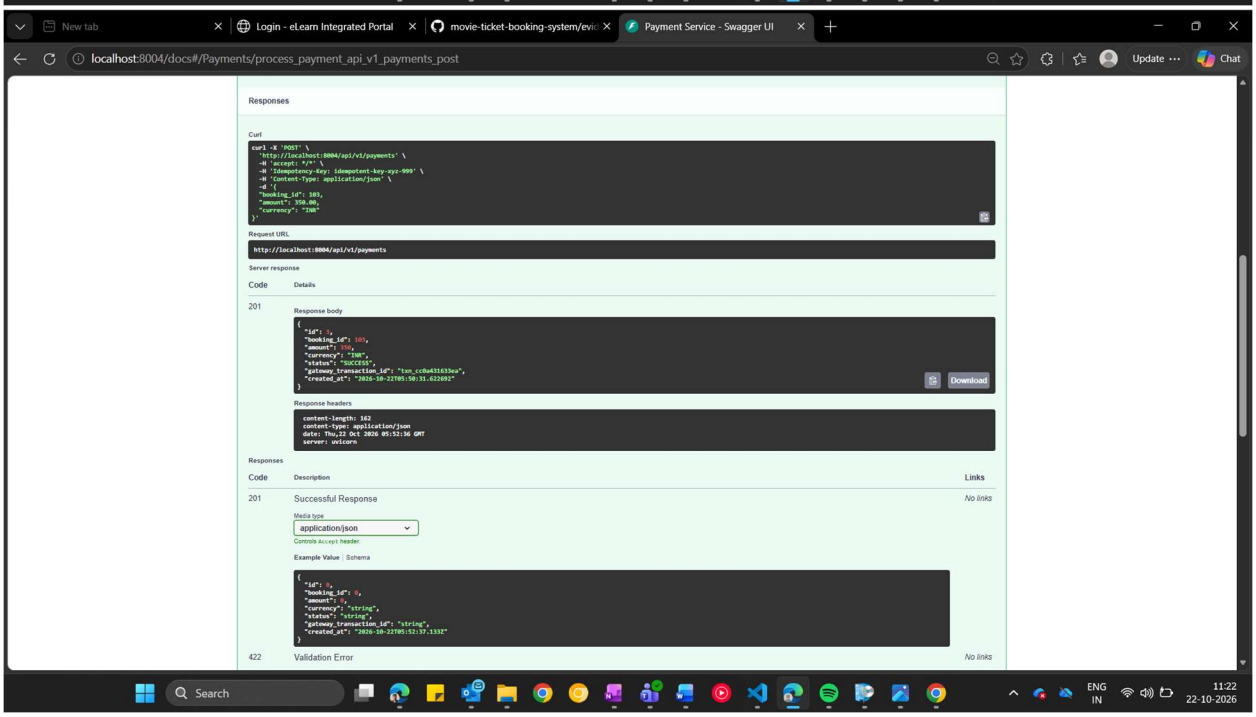


Figure 7. Duplicate Request Detection


New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/evic
Payment Service - Swagger UI
localhost:8004/docs#/Payments/process_payment_api_v1_payments_post
Update
Chat
Microservice responsible for processing transactions securely and idempotently.
Payments
POST
/api/v1/payments
Process Payment
Parameters
Cancel
Reset
Name
Description
Idempotency-Key
* required
string
(header)
idempotent-key-xyz-999
Request body
required
application/json
Edit Value | Schema
{
"booking_id": 103,
"amount": 350.00,
"currency": "INR"
}
Execute
Clear
Responses
Search
ENG
IN
11:20
22-10-2026


New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/evic
Payment Service - Swagger UI
localhost:8004/docs#/Payments/process_payment_api_v1_payments_post
Update
Chat
Execute
Clear
Responses
Curl
Request URL
http://localhost:8004/api/v1/payments
Server response
Code
Details
201
Response body
Download
Response headers
content-length: 162
content-type: application/json
date: Thu,23 Oct 2025 05:56:31 GMT
server: uvicorn
Responses
Code
Description
Links
201
Successful Response
No links
Media type
application/json
Example Value | Schema
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
],
"msg": "string",
"type": "string"
}
]
}
Search
ENG
IN
11:21
22-10-2026


New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/evic
Payment Service - Swagger UI
localhost:8004/docs#/Payments/process_payment_api_v1_payments_post
Update
Chat
Responses
Curl
curl -X 'POST' \
'http://localhost:8004/api/v1/payments' \
-H 'accept: */*' \
-H 'Idempotency-Key: idempotent-key-xyz-999' \
-H 'Content-Type: application/json' \
-d '{
"booking_id": 103,
"amount": 350.00,
"currency": "INR"
}'
Request URL
http://localhost:8004/api/v1/payments
Server response
Code
Details
201
Response body
{
"id": 3,
"booking_id": 103,
"amount": 450,
"currency": "INR",
"status": "SUCCESS",
"gateway_transaction_id": "txn_cc8a431633ea",
"created_at": "2026-10-22T05:50:31.622692"
}
Download
Response headers
content-length: 162
content-type: application/json
date: Thu,22 Oct 2026 05:51:17 GMT
server: uvicorn
Responses
Code
Description
Links
201
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": 0,
"booking_id": 0,
"amount": 0,
"currency": "string",
"status": "string",
"gateway_transaction_id": "string",
"created_at": "2026-10-22T05:51:17.983Z"
}
422
Validation Error
No links
Search
ENG
IN
11:22
22-10-2026


New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/evic
Payment Service - Swagger UI
localhost:8004/docs#/Payments/process_payment_api_v1_payments_post
Update
Chat
Responses
Curl
curl -X 'POST' \
'http://localhost:8004/api/v1/payments' \
-H 'accept: */*' \
-H 'Idempotency-Key: idempotent-key-xyz-999' \
-H 'Content-Type: application/json' \
-d '{
"booking_id": 103,
"amount": 350.00,
"currency": "INR"
}'
Request URL
http://localhost:8004/api/v1/payments
Server response
Code
Details
201
Response body
{
"id": 3,
"booking_id": 103,
"amount": 350,
"currency": "INR",
"status": "SUCCESS",
"gateway_transaction_id": "txn_cc8a431633ea",
"created_at": "2026-10-22T05:50:31.622692"
}
Download
Response headers
content-length: 162
content-type: application/json
date: Thu,22 Oct 2026 05:52:36 GMT
server: uvicorn
Responses
Code
Description
Links
201
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": 0,
"booking_id": 0,
"amount": 0,
"currency": "string",
"status": "string",
"gateway_transaction_id": "string",
"created_at": "2026-10-22T05:52:37.133Z"
}
422
Validation Error
No links
Search
ENG
IN
11:22
22-10-2026

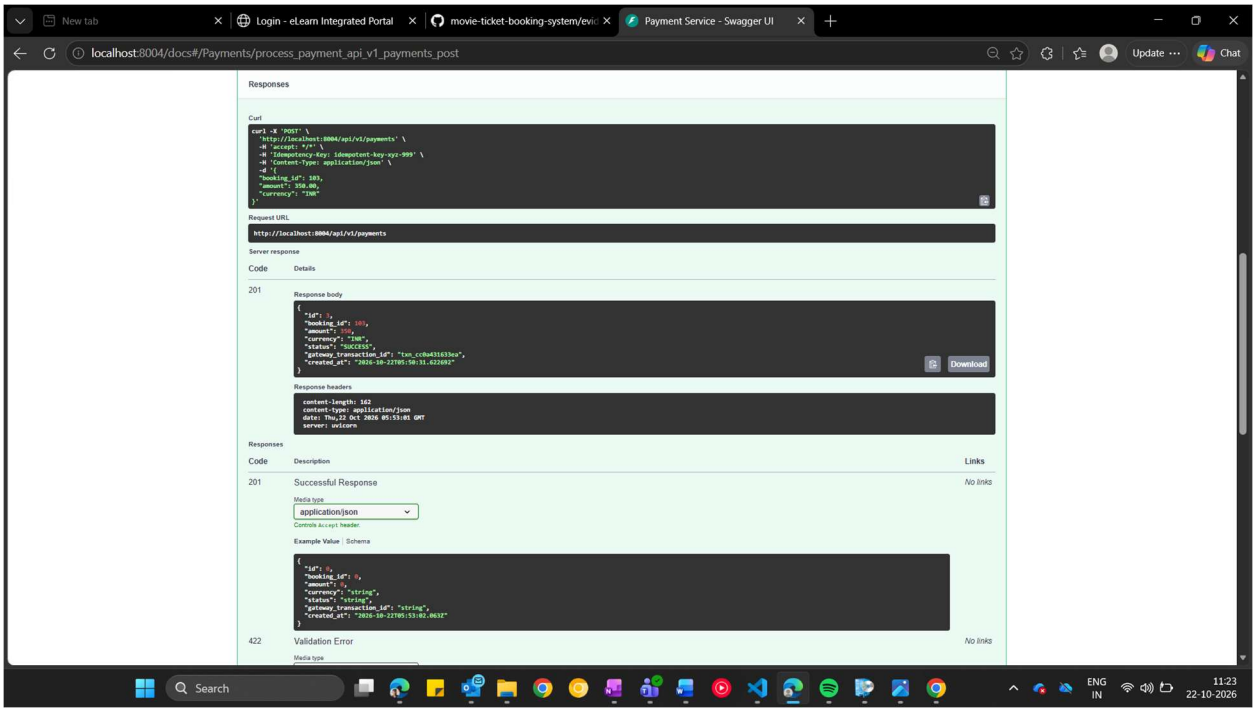


Figure 8. Payment Service Idempotency Validation

Test Result

- PAY123 → Processed
- PAY123 → Duplicate
- PAY123 → Duplicate

Conclusion

Only a single payment record is created even when identical requests are repeatedly submitted.

Limitation

The implementation assumes trusted idempotency key generation by the client. Production systems may require additional key expiration and lifecycle management.

Challenge 3C – Saga Pattern

Objective

Implement distributed transaction handling across services.

Problem

A seat may already be reserved when a payment subsequently fails, causing data inconsistency across services.

Design Choice

The Booking Service acts as a Saga Orchestrator and coordinates the distributed workflow.

Alternative Considered

Saga choreography was considered but rejected because orchestration provides simpler control flow and easier debugging for educational and demonstration purposes.

Successful Booking Saga

Screenshots

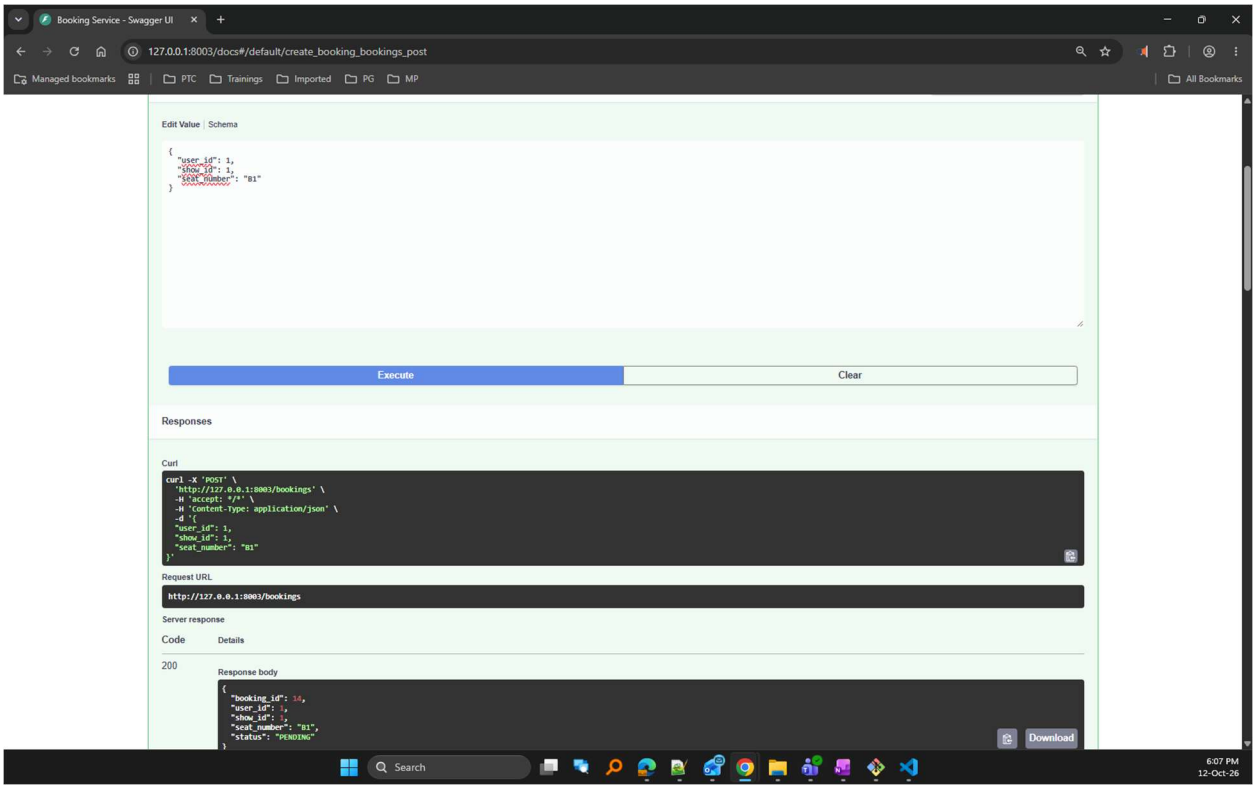


Figure 9. Successful Booking Saga

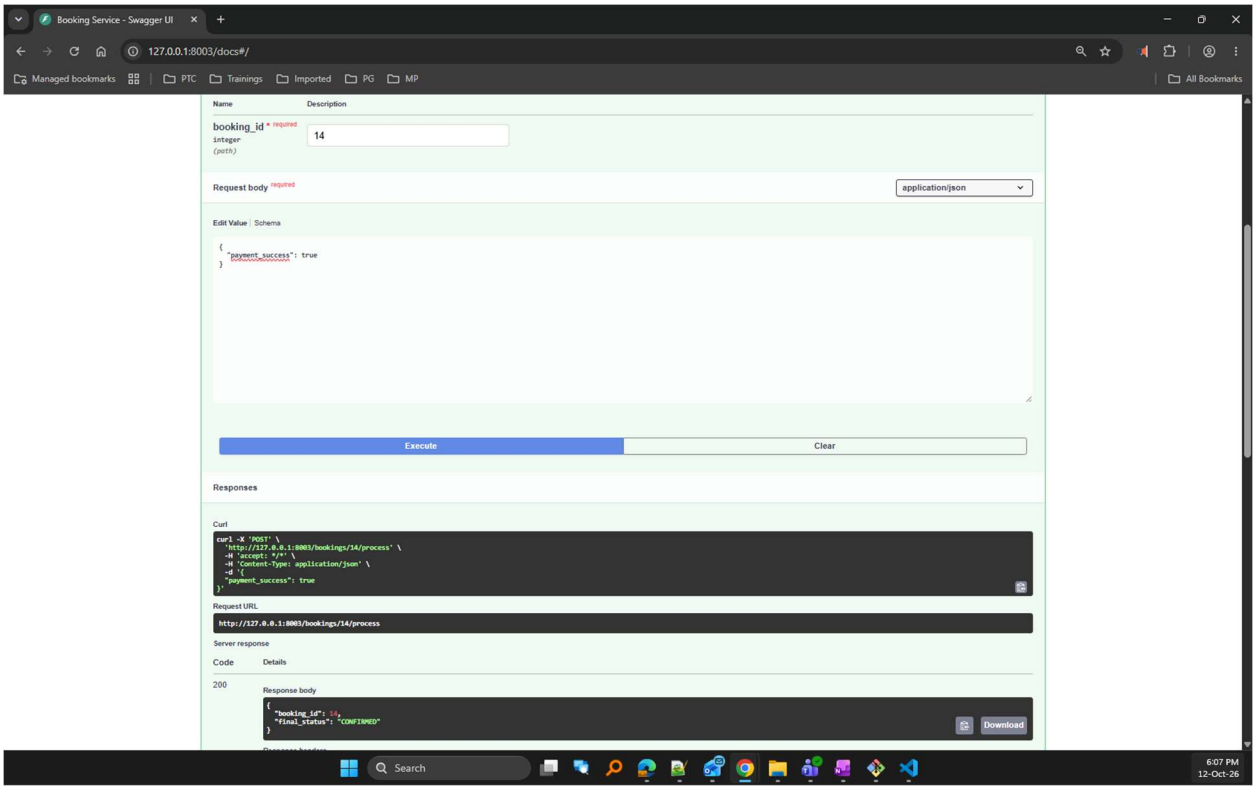


Figure 10. Confirmed Booking State

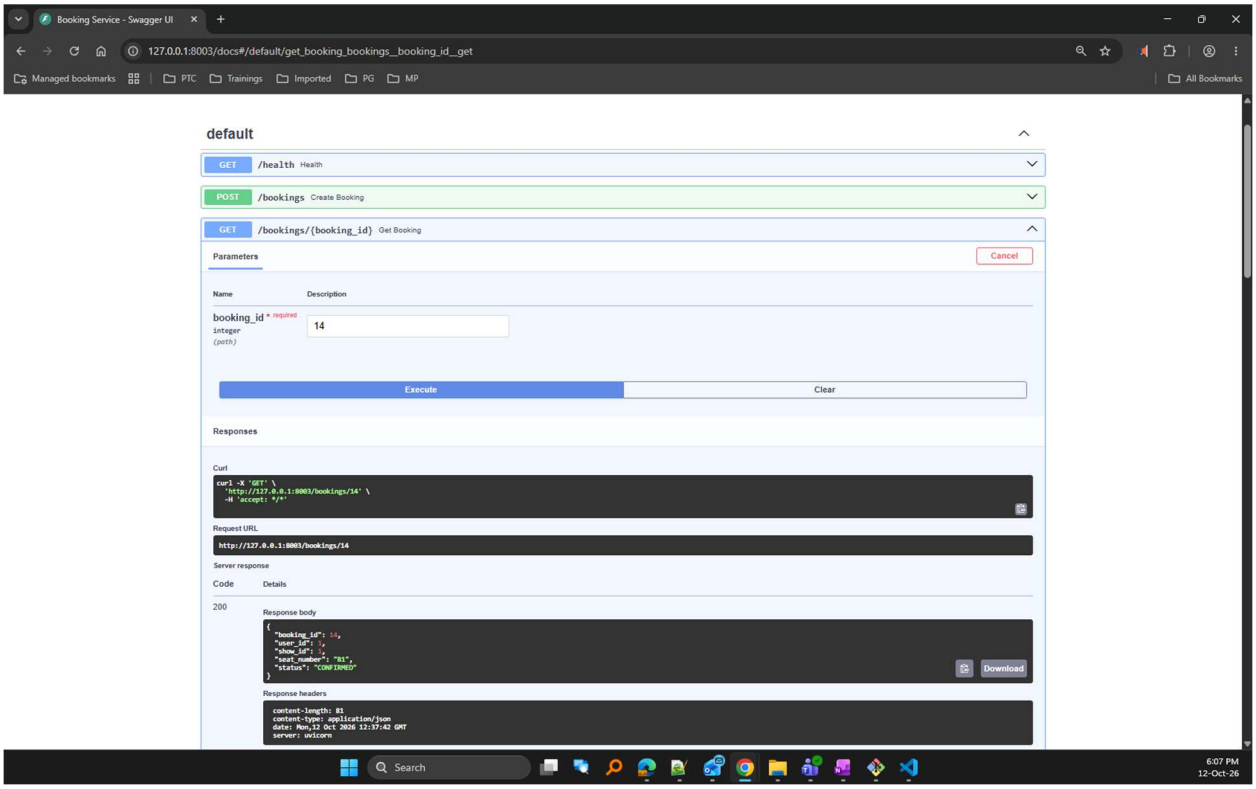


Figure 11. Confirmed Booking State

Flow

PENDING

↓

RESERVED

↓

CONFIRMED

Notification:

- Booking confirmed

Compensation Workflow

Screenshots

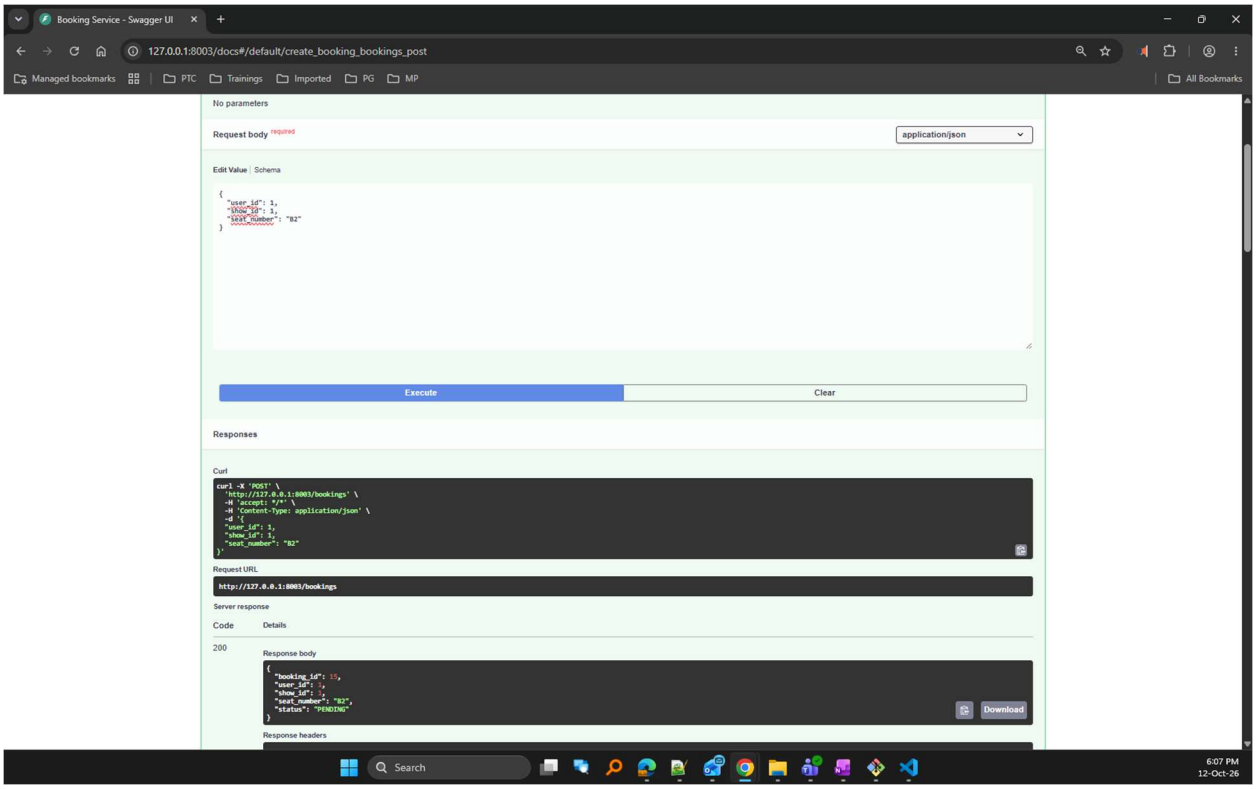


Figure 12. Payment Failure Compensation

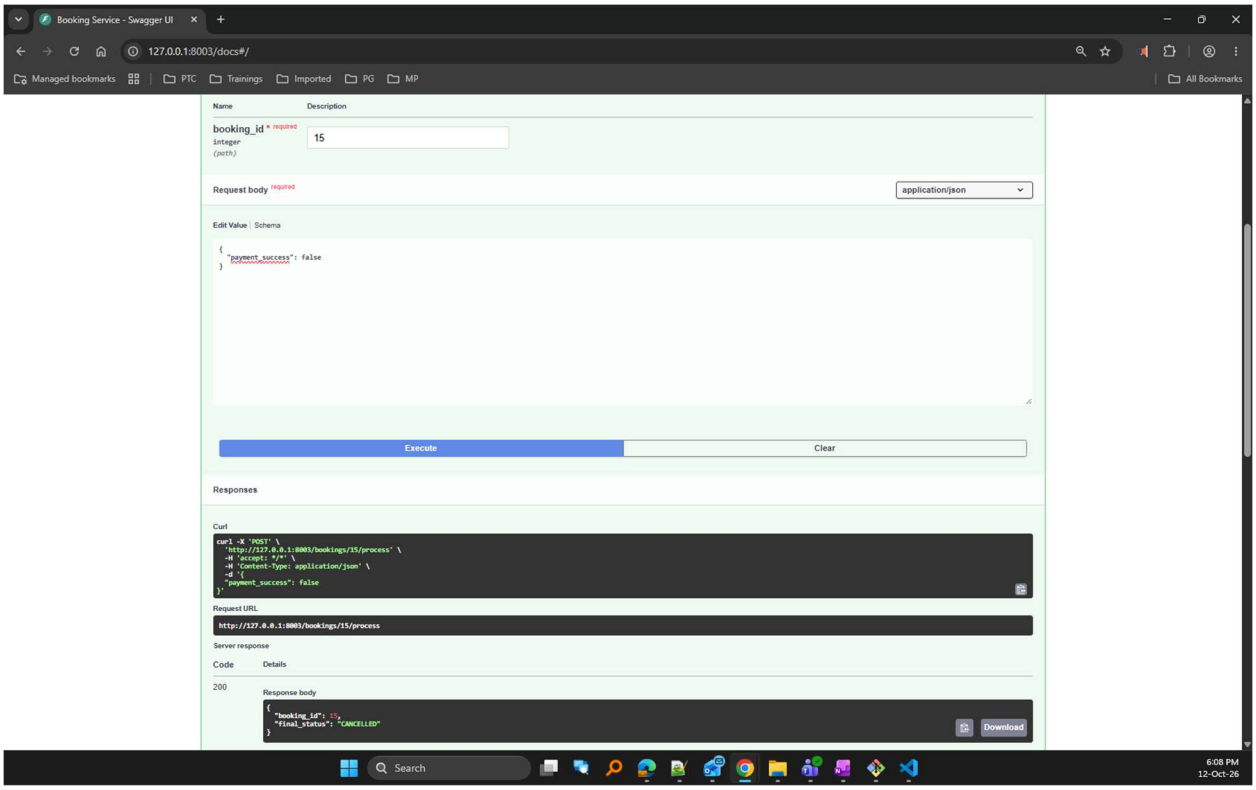


Figure 13. Cancelled Booking State

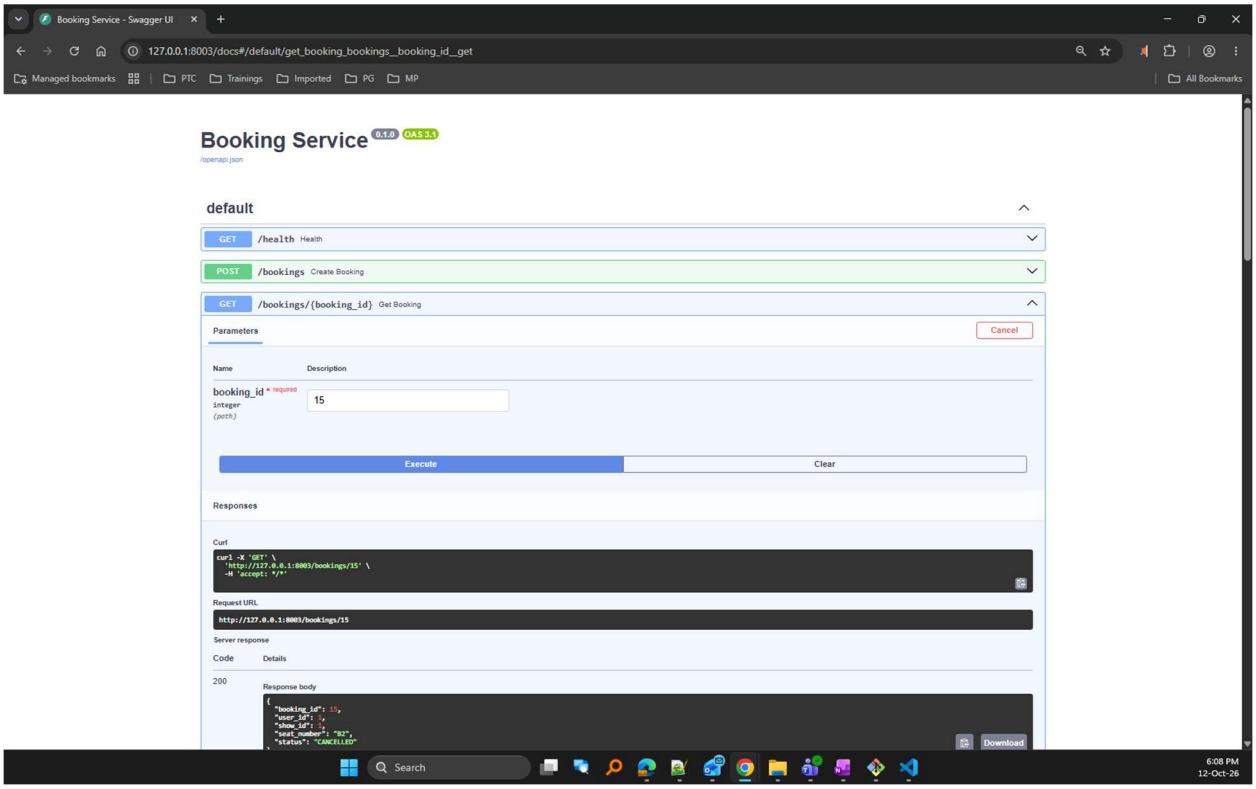


Figure 14. Cancelled Booking State

Flow

PENDING
↓
RESERVED
↓
PAYMENT_FAILED
↓
CANCELLED

Compensation:

- Seat released
- Booking cancelled

Notification:

- Booking cancelled because payment failed

Evidence

End-to-end integration testing verified that successful payments lead to confirmed bookings while failed payments trigger compensation actions and seat release.

Limitation

The payment gateway is simulated. Production implementations would require integration with a real payment provider and stronger failure recovery mechanisms.

---

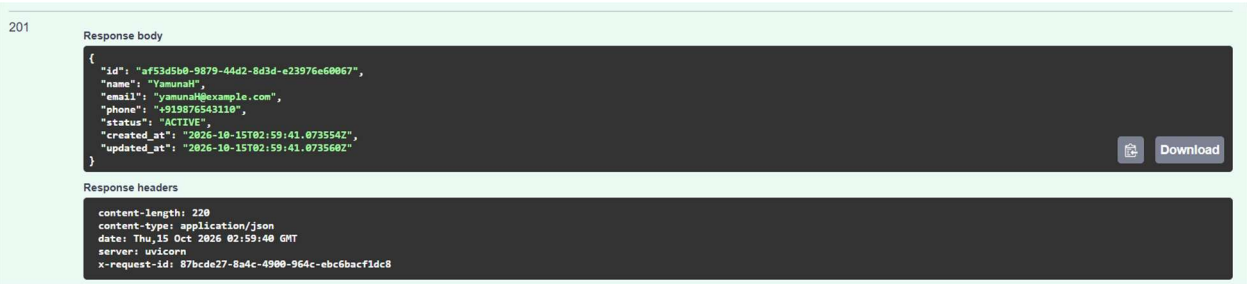
7. Testing

Testing Strategy

The system was validated using:

- Swagger UI testing
- Postman-based API testing
- Automated pytest unit tests
- Concurrency testing scripts
- Idempotency testing
- End-to-end integration testing through integration_demo.py

User Service



Code | Details

200

Response body


```
{
  "id": "bfe1fce8-885d-48a2-986d-5c385a14efb9",
  "name": "Yamuna",
  "email": "yamuna@example.com",
  "phone": "+919876543210",
  "status": "ACTIVE",
  "created_at": "2026-10-15T02:55:39.196720",
  "updated_at": "2026-10-15T02:55:39.196727"
}
```


Download

Response headers

```
content-length: 216
content-type: application/json
date: Thu,15 Oct 2026 14:05:52 GMT
server: uvicorn
x-request-id: 27e16ec4-7f68-4143-8ee7-f49b35a6375c
```

Responses

Code | Details

422 Error: Unprocessable Content

Response body


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [
      {
        "field": "phone",
        "message": "Value error, phone must be 10-15 digits, optionally starting with '+'"
      }
    ],
    "request_id": "d7dba739-4f82-4067-b430-638d5118ea3a"
  }
}
```


Download

Response headers

```
content-length: 239
content-type: application/json
date: Thu,15 Oct 2026 03:07:34 GMT
server: uvicorn
x-request-id: d7dba739-4f82-4067-b430-638d5118ea3a
```

Responses

Name | Description

user_id * required
string
(path)

abc

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8001/api/v1/users/abc' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8001/api/v1/users/abc
```

Server response

Code | Details

404 Error: Not Found

Response body


```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "User 'abc' not found",
    "details": [],
    "request_id": "8bf728ec-ec2d-4ca5-b947-8c553b2baccc"
  }
}
```


Download

Response headers

```
content-length: 133
content-type: application/json
date: Thu,15 Oct 2026 14:11:47 GMT
server: uvicorn
x-request-id: 8bf728ec-ec2d-4ca5-b947-8c553b2baccc
```

Responses



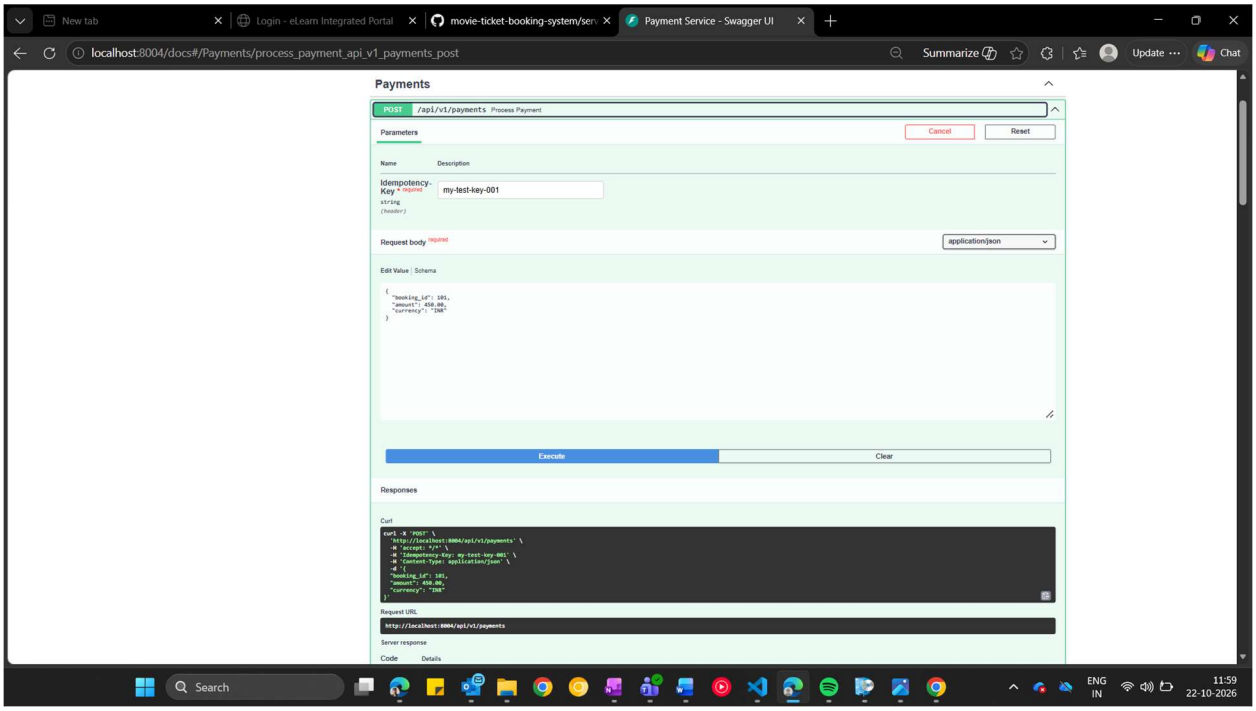
Figure 15. User Service Testing

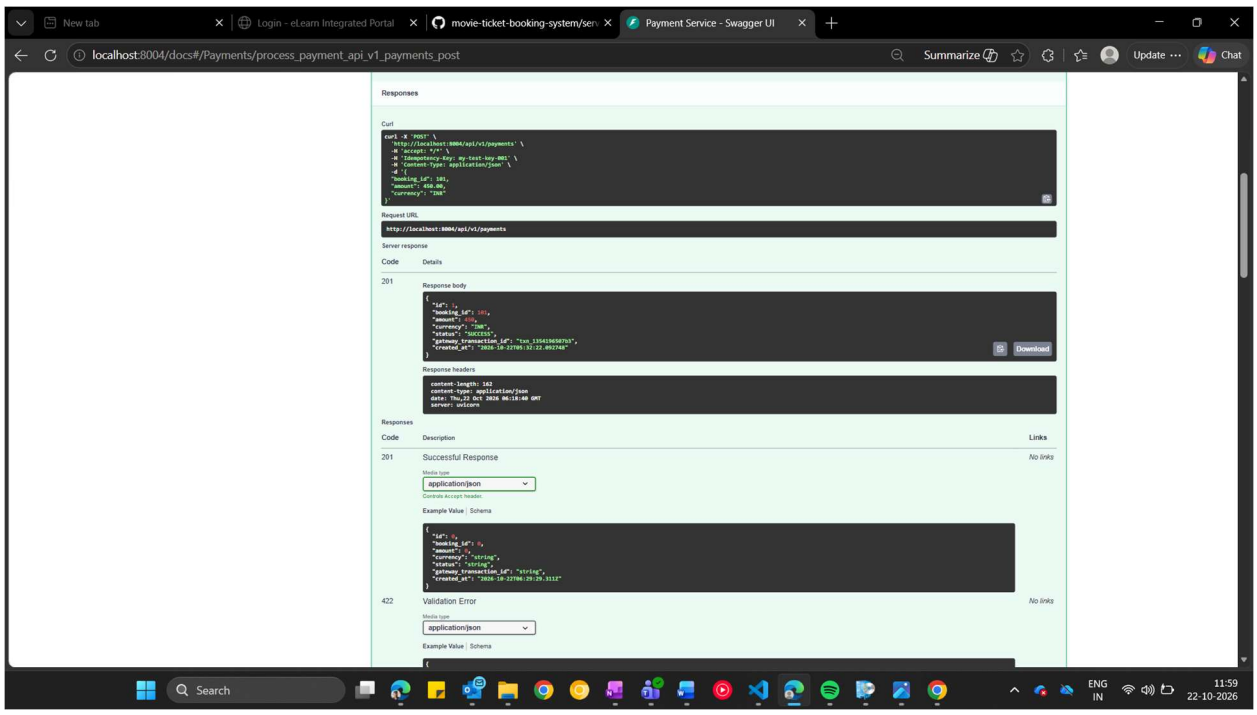
Result

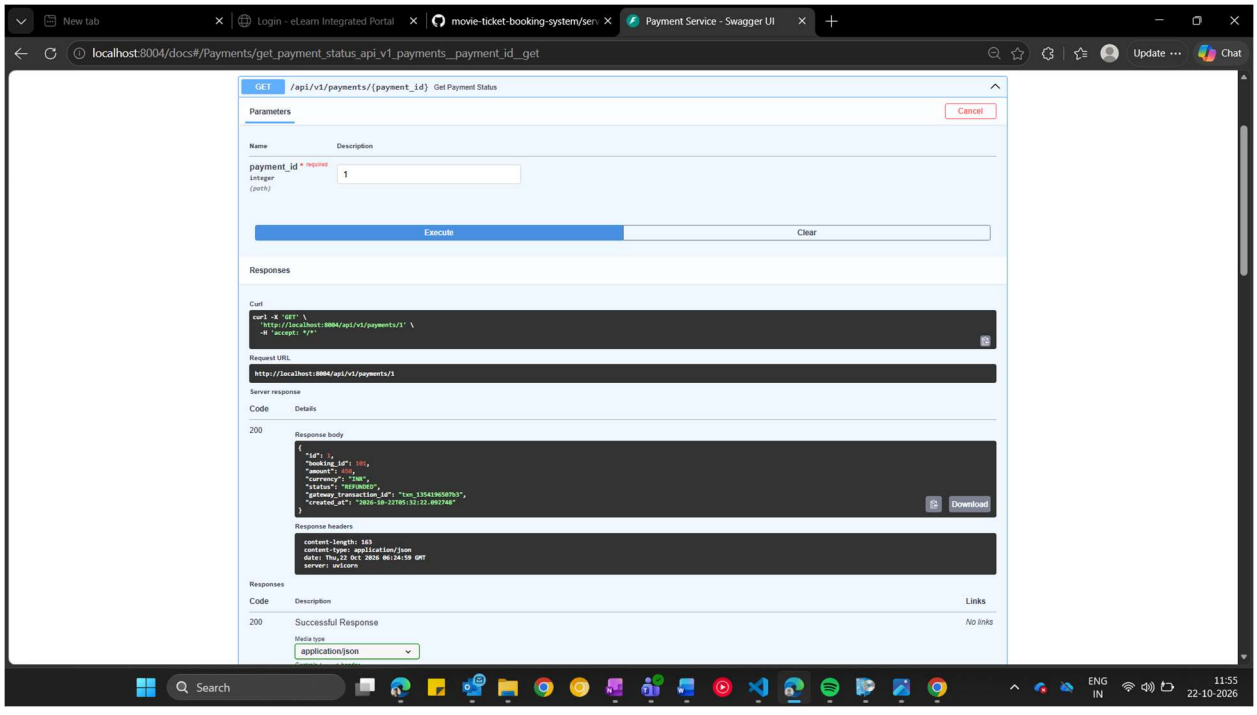
- 20 tests passed
- 0 failed

Payment Service

Screenshots




New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/ser
Payment Service - Swagger UI
localhost:8004/docs#/Payments/process_payment_api_v1_payments_post
Summarize
Update
Chat
Responses
Curl
curl -X 'POST' \
'http://localhost:8004/api/v1/payments' \
-H 'accept: */*' \
-H 'Idempotency-Key: my-test-key-001' \
-H 'Content-Type: application/json' \
-d '{
"booking_id": 101,
"amount": 450.00,
"currency": "INR"
}'
Request URL
http://localhost:8004/api/v1/payments
Server response
Code
Details
201
Response body
{
"id": 1,
"booking_id": 101,
"amount": 450,
"currency": "INR",
"status": "SUCCESS",
"gateway_transaction_id": "txn_1354196507b3",
"created_at": "2026-10-22T05:32:22.092748"
}
Download
Response headers
content-length: 162
content-type: application/json
date: Thu,22 Oct 2026 06:18:40 GMT
server: uvicorn
Responses
Code
Description
Links
201
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"id": 0,
"booking_id": 0,
"amount": 0,
"currency": "string",
"status": "string",
"gateway_transaction_id": "string",
"created_at": "2026-10-22T06:29:29.311Z"
}
422
Validation Error
No links
Media type
application/json
Example Value | Schema
Search
ENG
IN
11:59
22-10-2026


New tab
Login - eLearn Integrated Portal
movie-ticket-booking-system/ser
Payment Service - Swagger UI
localhost:8004/docs#/Payments/get_payment_status_api_v1_payments__payment_id__get
Update
Chat
GET
/api/v1/payments/{payment_id}
Get Payment Status
Parameters
Cancel
Name
Description
payment_id * required
Integer
(path)
1
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8004/api/v1/payments/1' \
-H 'accept: */*'
Request URL
http://localhost:8004/api/v1/payments/1
Server response
Code
Details
200
Response body
{
"id": 1,
"booking_id": 101,
"amount": 450,
"currency": "INR",
"status": "REFUNDED",
"gateway_transaction_id": "txn_1354196507b3",
"created_at": "2026-10-22T05:32:22.092748"
}
Download
Response headers
content-length: 163
content-type: application/json
date: Thu,22 Oct 2026 06:24:59 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Search
ENG
IN
11:55
22-10-2026

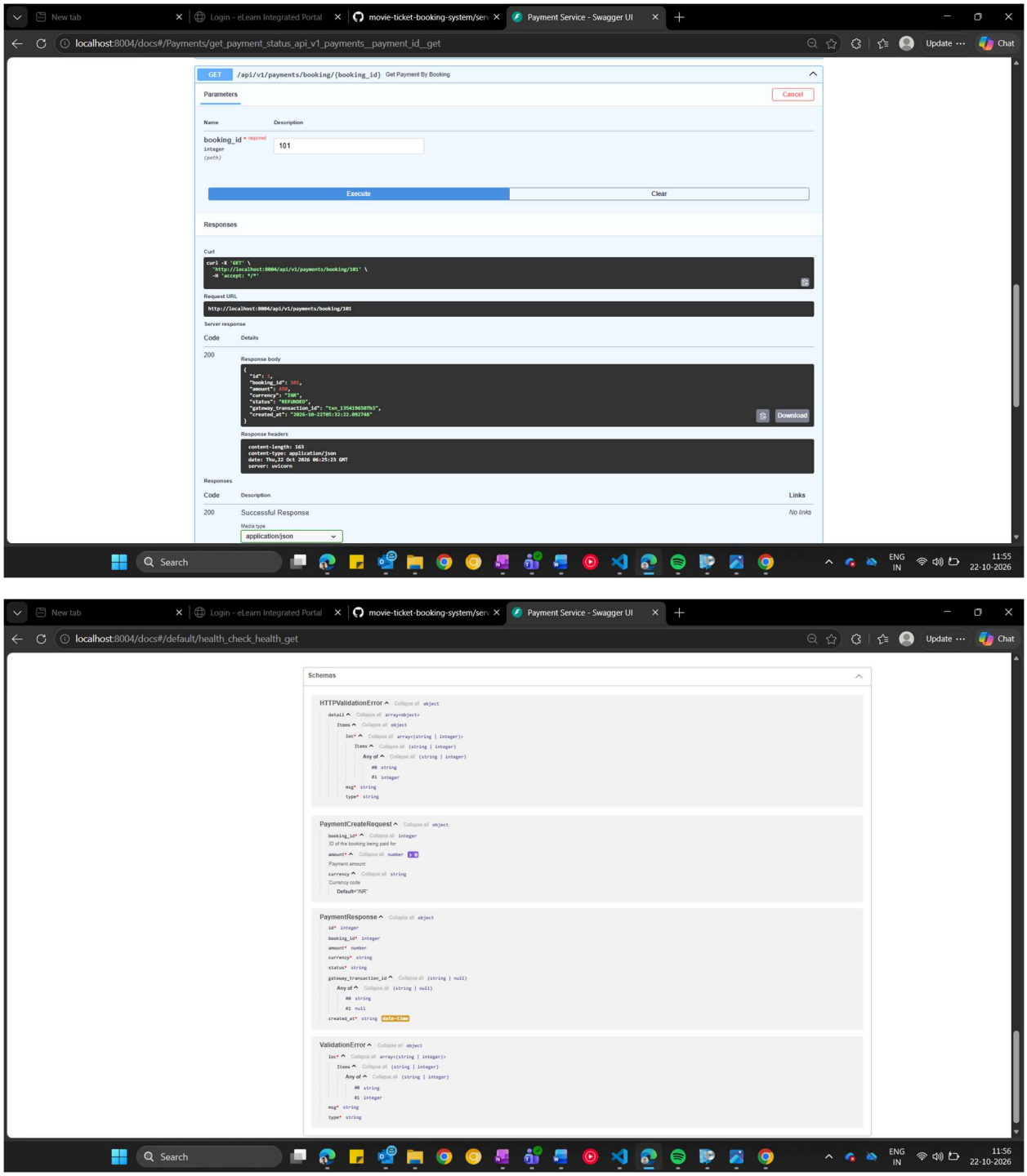


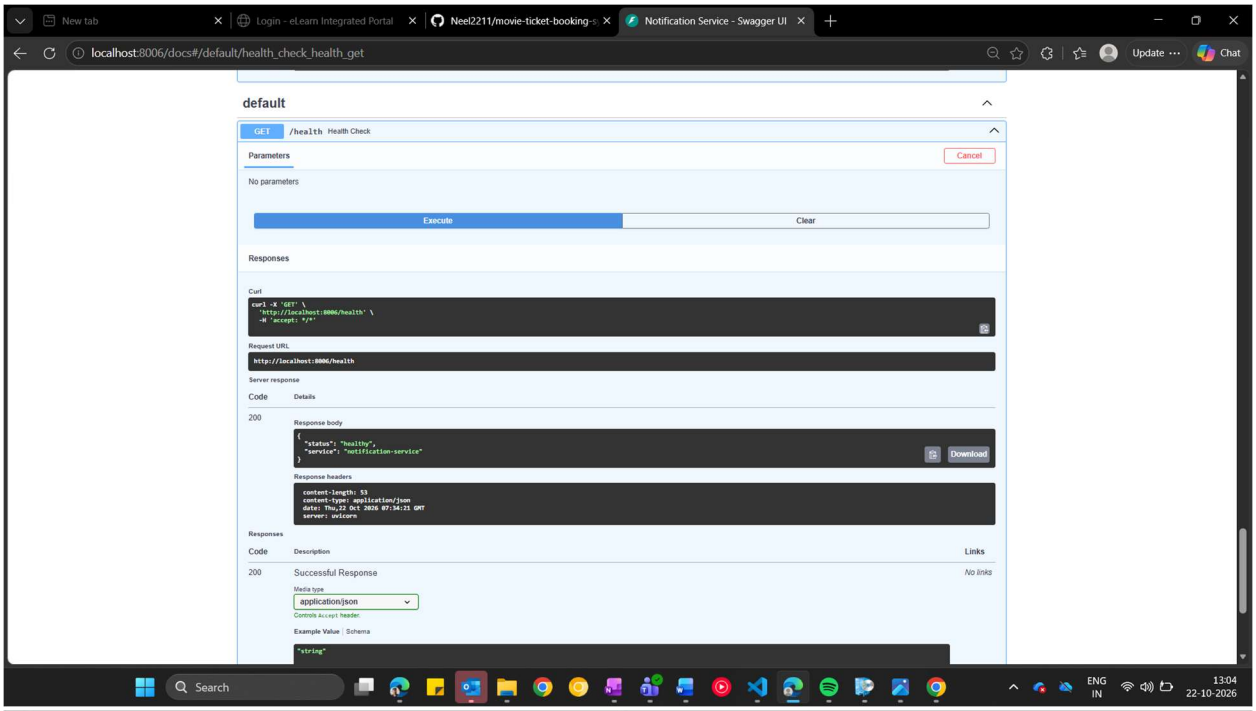
Figure 16. Payment Service Testing

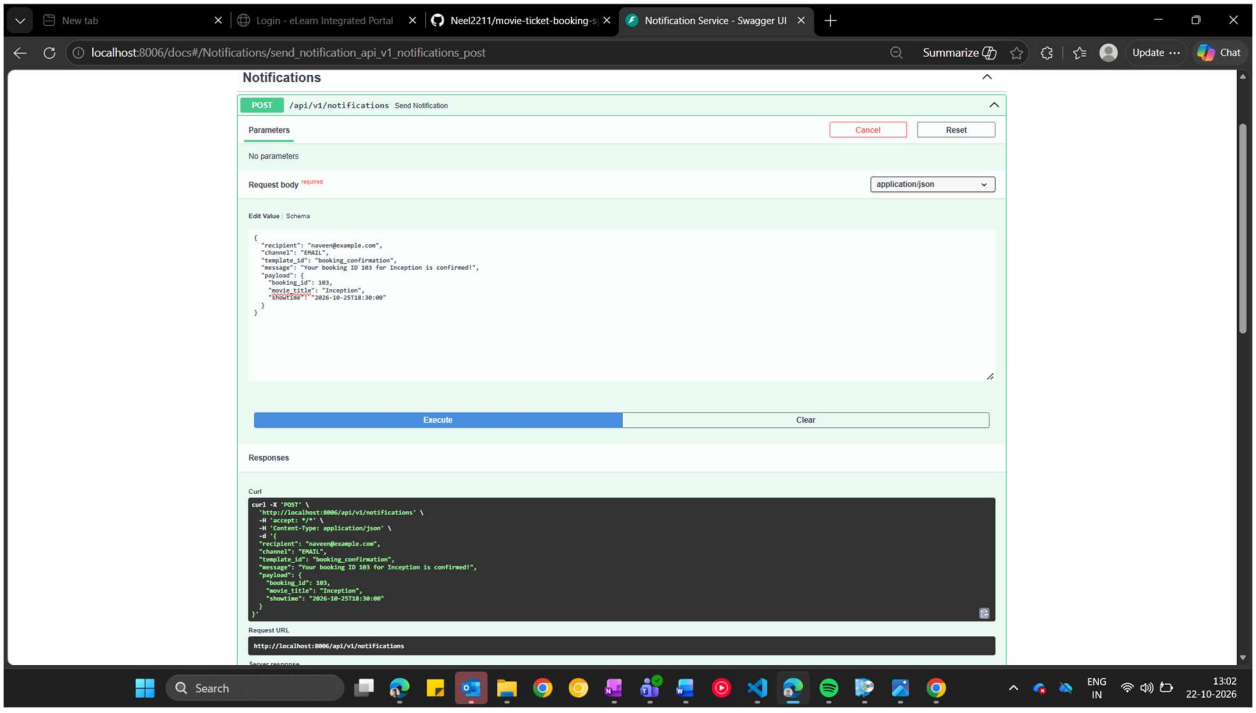
Result

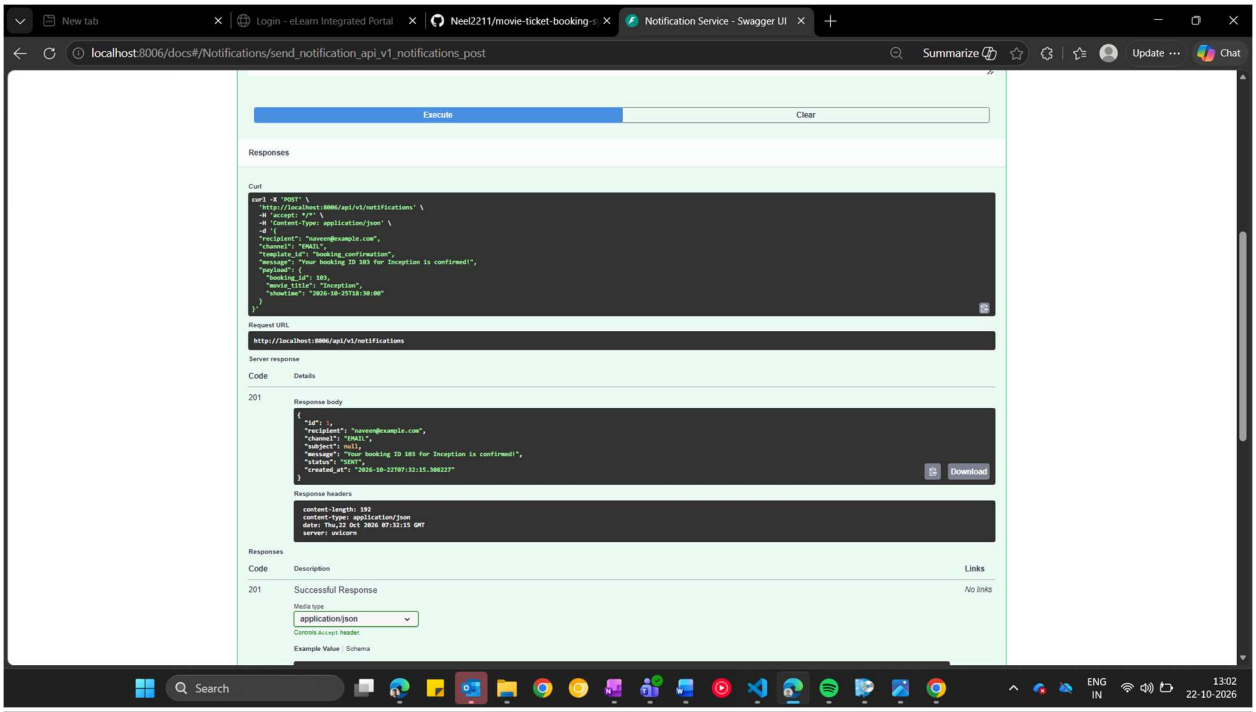
- 7 tests passed
- 0 failed

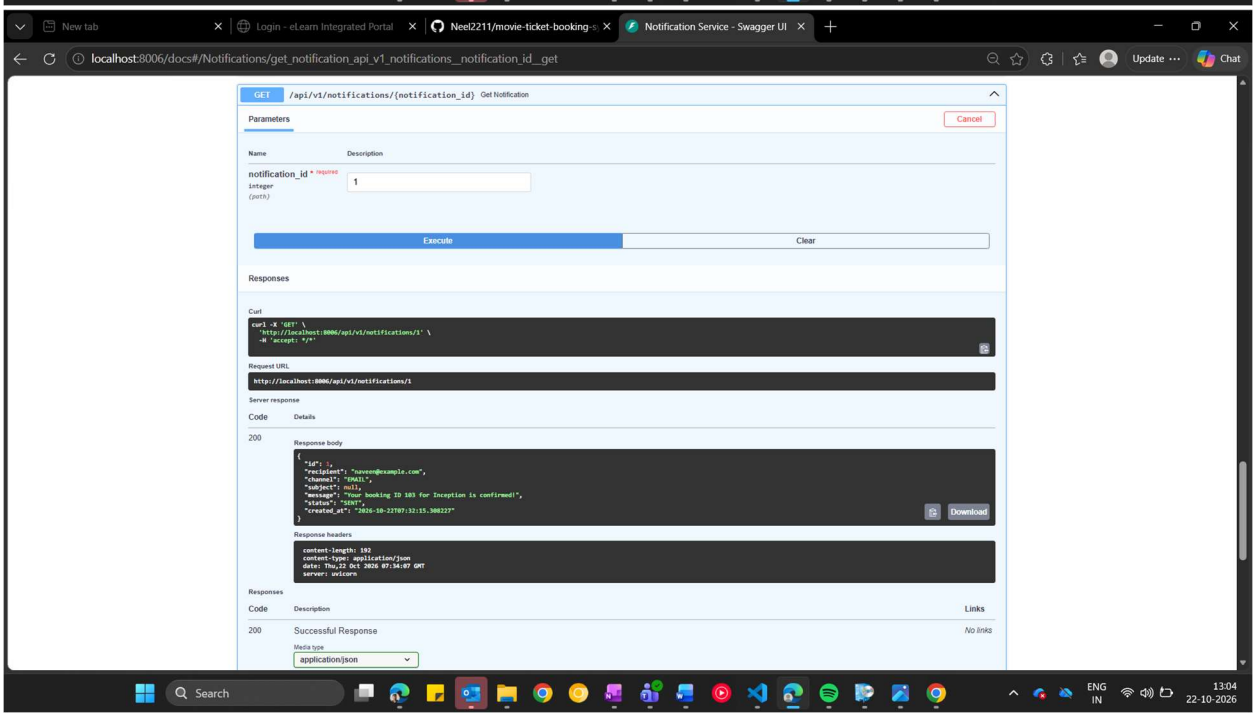
Notification Service

Screenshots






New tab
Login - eLearn Integrated Portal
Neel2211/movie-ticket-booking-
Notification Service - Swagger UI
localhost:8006/docs#/Notifications/send_notification_api_v1_notifications_post
Summarize
Update
Chat
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://localhost:8006/api/v1/notifications' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"recipient": "naveen@example.com",
"channel": "EMAIL",
"template_id": "booking_confirmation",
"message": "Your booking ID 103 for Inception is confirmed!",
"payload": {
"booking_id": 103,
"movie_title": "Inception",
"showtime": "2026-10-25T18:30:00"
}
}'
Request URL
http://localhost:8006/api/v1/notifications
Server response
Code
Details
201
Response body
{
"id": 1,
"recipient": "naveen@example.com",
"channel": "EMAIL",
"subject": null,
"message": "Your booking ID 103 for Inception is confirmed!",
"status": "SENT",
"created_at": "2026-10-22T07:32:15.308227"
}
Download
Response headers
content-length: 192
content-type: application/json
date: Thu,22 Oct 2026 07:32:15 GMT
server: uvicorn
Responses
Code
Description
Links
201
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
Search
ENG
IN
13:02
22-10-2026


New tab
Login - eLearn Integrated Portal
Neel2211/movie-ticket-booking-
Notification Service - Swagger UI
localhost:8006/docs#/Notifications/get_notification_api_v1_notifications__notification_id__get
Update
Chat
GET
/api/v1/notifications/{notification_id}
Get Notification
Parameters
Cancel
Name
Description
notification_id * required
integer
(path)
1
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8006/api/v1/notifications/1' \
-H 'accept: */*'
Request URL
http://localhost:8006/api/v1/notifications/1
Server response
Code
Details
200
Response body
{
"id": 1,
"recipient": "naveen@example.com",
"channel": "EMAIL",
"subject": null,
"message": "Your booking ID 103 for Inception is confirmed!",
"status": "SENT",
"created_at": "2026-10-22T07:32:15.308227"
}
Download
Response headers
content-length: 192
content-type: application/json
date: Thu,22 Oct 2026 07:34:07 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Search
ENG
IN
13:04
22-10-2026

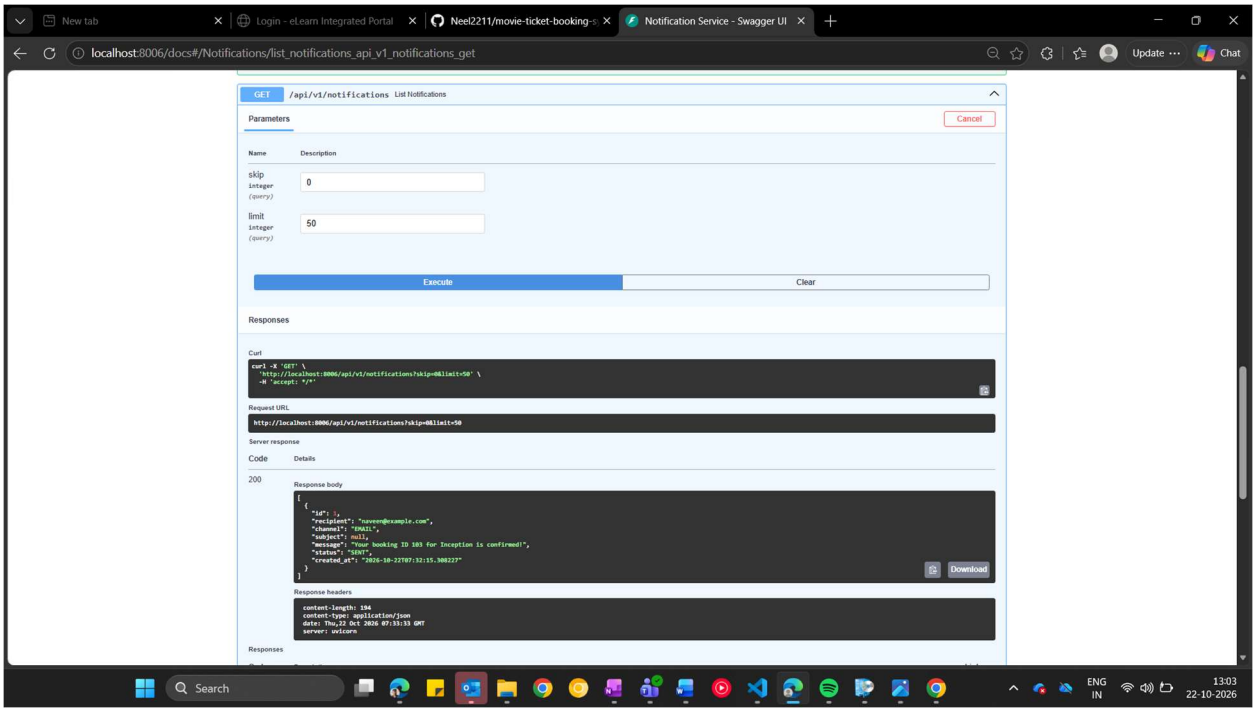


Figure 17. Notification Service Testing

End-to-End Integration Testing

Objective

Validate communication among all five services.

Services Running

- User Service 8001
- Show Service 8002
- Booking Service 8003
- Payment Service 8004
- Notification 8006

Integration Verification

Execution:

- python integration_demo.py

Validated:

- User creation
- Booking creation
- Seat reservation
- Double booking prevention

- Payment failure compensation
- Notification generation
- User business rules

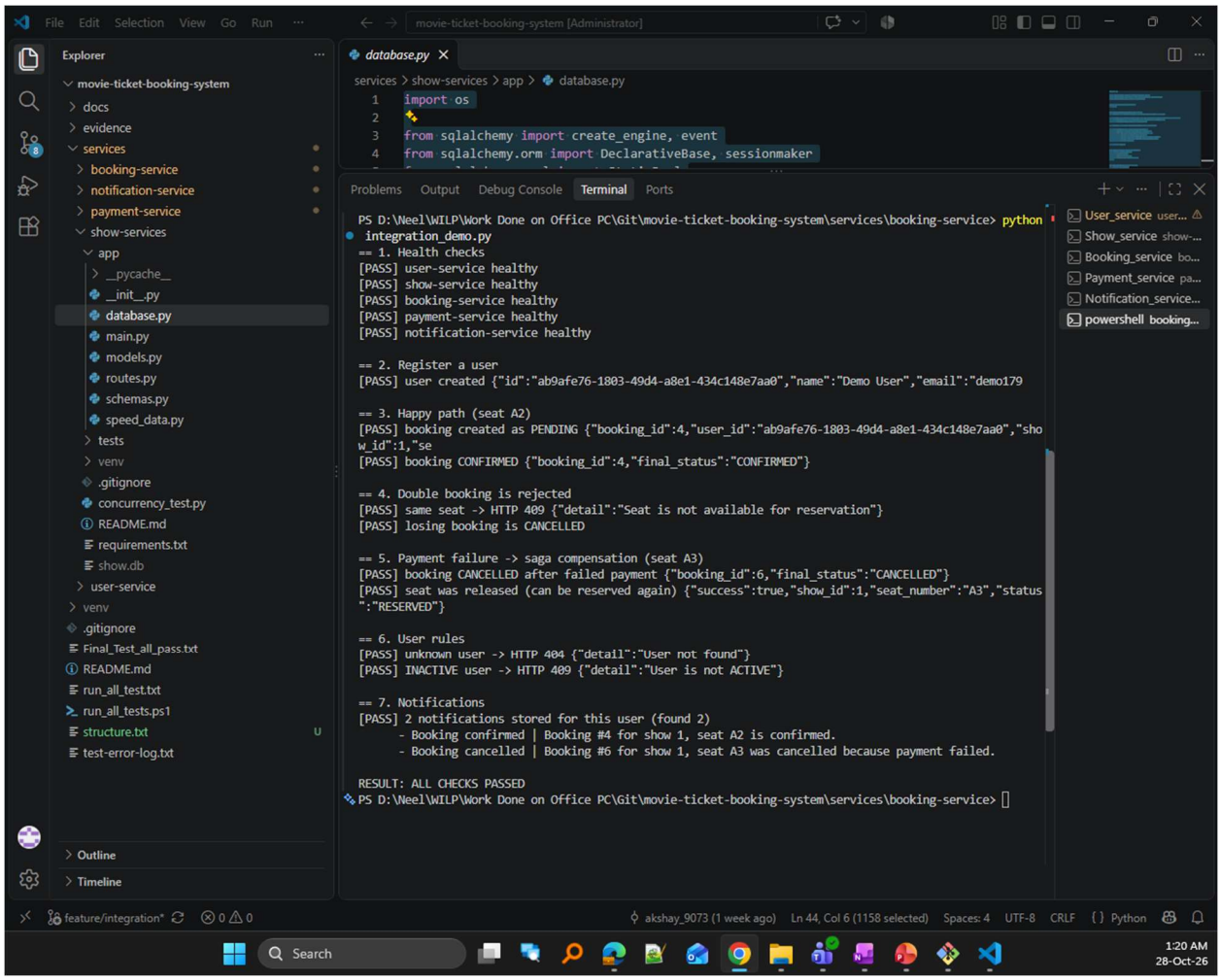


Figure 18. End-to-End Integration Validation

Outcome

- RESULT: ALL CHECKS PASSED

---

8. Design Decisions and Trade-Offs

Decision 1: Saga Orchestration vs Choreography

Selected:

Orchestration

Reason:

Simpler coordination and debugging.

Decision 2: SQLite vs MySQL

Selected:

SQLite

Reason:

Minimal setup complexity and sufficient for demonstration.

Decision 3: REST vs gRPC

Selected:

REST

Reason:

Built-in OpenAPI support and easier testing using Swagger/Postman.

---

9. Limitations and Future Work

Limitations

- Notifications are mocked.
- Payment gateway is simulated.
- Authentication is not implemented.
- SQLite limits concurrent scalability.

Future Work

- JWT Authentication
- Containerization using Docker
- Kubernetes deployment
- Asynchronous messaging using Kafka/RabbitMQ
- Real payment gateway integration

---

10. Links to Deliverables

GitHub Repository

https://github.com/Neel2211/movie-ticket-booking-system

Demo Video

https://drive.google.com/file/d/1QGtG_73qK3UGgOQIvoZlCURTHOuAWsj2/view?usp=sharing

Supporting Documents

https://drive.google.com/drive/folders/1vKiZQ2M82bdlyzjHHlD6VDuAoxwchvmg?usp=sharing

---

11. Conclusion

The Movie Ticket Booking System was successfully implemented using a microservices architecture. The solution demonstrates database-per-service design, concurrency control, idempotent request processing, and Saga-based transaction management. Integration testing verified successful interaction among all five microservices, with all test scenarios passing successfully. The final implementation satisfies all assignment requirements and demonstrates the practical application of scalable service design principles.

Appendix A – Contribution Table

Member	Contribution
Gujar Neel Lukesh	Booking Service implementation, booking state machine, Saga orchestration, integration testing, report preparation
Yamuna P Hegade	User Service implementation, validation rules, lifecycle management, testing
Bhandari Akshay Sunil	Show Service implementation, seat reservation logic, concurrency control, Integration testing
Naveen Venkat P	Payment Service implementation, idempotency handling, compensation workflows
Shared	Notification Service, end-to-end testing, final integration

---

Appendix B – Individual Reflection

B.1 Gujar Neel Lukesh

I was responsible for the implementation of the Booking Service and acted as the integration coordinator for the project. My primary tasks included developing booking APIs, implementing booking lifecycle management, designing the booking state machine, and coordinating service integration. I also implemented the Saga orchestration workflow to ensure consistency across distributed services when payment failures occurred.

The most challenging aspect of the project was handling distributed transactions while maintaining consistency between services. Implementing the Saga compensation workflow required careful consideration of state transitions and failure scenarios. I designed the workflow so that failed payments automatically trigger seat release and booking cancellation. Through this work I gained practical experience in microservice integration, service orchestration, and fault handling. If the project were extended, I would explore event-driven communication and asynchronous messaging to improve scalability and resilience.

This project improved my understanding of microservices architecture and distributed systems design.

B.2 Yamuna P Hegade

I was responsible for the User Service. My contribution included implementing user creation, retrieval, activation, deactivation, validation logic, and lifecycle management. I designed and enforced business rules to ensure that only valid and active users could participate in the booking process.

The most challenging part of my work was implementing business-rule validation and handling different user states consistently. I focused on ensuring that inactive users could not perform restricted actions and that duplicate user records were prevented. This project improved my understanding of API design, service ownership, validation strategies, and reliable error handling. In a future version of the application, I would enhance the User Service by introducing authentication, authorization, and role-based access control.

This project improved my understanding of microservices architecture and distributed systems design.

B.3 Bhandari Akshay Sunil

I was responsible for the Show Service. My work involved implementing show management, seat inventory management, seat reservation, seat release functionality, and concurrency handling. The service was designed to ensure that multiple users could not reserve the same seat simultaneously.

The most difficult aspect of this service was implementing concurrency control in a simple yet reliable manner. I designed seat reservation logic using atomic database operations so that only one request could successfully reserve a seat while competing requests were rejected. Through this implementation I learned how shared resources must be protected in distributed systems. Given more time, I would enhance the service with dynamic seat layouts, multiple screens, and richer show-management functionality.

This project improved my understanding of microservices architecture and distributed systems design.

B.4 Naveen Venkat P

I was responsible for the Payment Service. My contribution included payment processing APIs, idempotency handling, payment retrieval operations, and support for Saga compensation workflows. I implemented logic to ensure that repeated payment requests with the same idempotency key were processed only once.

The most challenging aspect of my work was implementing idempotent payment processing and ensuring that duplicate requests did not result in duplicate payment records. I also contributed to compensation handling by supporting the refund and failure scenarios required by the Saga workflow. This work improved my understanding of transaction consistency, payment reliability, and distributed-system error handling. If the system were extended further, I would integrate a real payment gateway and add more advanced fraud-prevention mechanisms.

This project improved my understanding of microservices architecture and distributed systems design.

Appendix C – AI Use Declaration

This project made use of AI-assisted tools, including Microsoft Copilot, to support activities such as report structuring, documentation drafting, architecture discussions, code review assistance, and technical writing improvement.

All implementation, debugging, testing, integration verification, challenge execution, evidence collection, and final validation activities were performed and reviewed by the project team members. AI-generated suggestions were evaluated, modified where necessary, and validated through actual implementation and testing before inclusion in the final submission.
